

System Design Document

1. Overview

The E-Commerce Shopping Cart System is a command-line Java application that provides a complete workflow for managing an online shopping session. It features inventory reading, cart management, coupon application, tax calculation, and data persistence.

2. Architecture

The system uses a straightforward monolithic architecture consisting of distinct conceptual layers:

- **Presentation / CLI Layer (`Main.java`):** Coordinates user inputs, displays menus, handles exceptions, and provides formatted output to the console.
- **Business Logic Layer (`Cart.java` , `CartItem.java` , `Product.java` , `Coupon.java`):** Contains the core logic for calculating totals, enforcing stock constraints, and verifying coupon validities.
- **Data Access Layer (`FileHandler.java`):** Abstracted file I/O layer that reads and writes domain objects (`Product` , `Coupon`) to and from CSV files.
- **Utility Layer (`Utils.java`):** Provides common helper methods (e.g., date parsing, rounding).

3. Core Classes and Entities

Product

Represents an item in the catalog.

- **Properties:** ID, name, price, stock quantity.
- **Responsibilities:** Managing its own stock decrementing safely.

Cart

Manages the current user's shopping session.

- **Properties:** List of `CartItem` objects, currently applied `Coupon` .

- **Responsibilities:** Adding/removing/updating items, verifying inventory availability, calculating subtotal, tax, discount, and grand total. Empties itself after checkout.

CartItem

Wrapper/Decorator for a `Product` inside a `Cart` .

- **Properties:** Reference to a `Product` , quantity.
- **Responsibilities:** Calculating total price for the specific item row.

Coupon

Represents a discount voucher.

- **Properties:** Code, DiscountType (FLAT, PERCENTAGE), discount value, expiry date.
- **Responsibilities:** Validating against current date, applying discount logic dynamically based on its `DiscountType` .

FileHandler

Static utility class.

- **Responsibilities:** Parses CSV data to instantiate Maps of `Product` and `Coupon` . Saves updated `Product` inventory back to CSV after successful checkout.

4. Design Patterns Used

- **Strategy Pattern (Implicit):** The `Coupon` class implements different discount algorithms (Flat vs. Percentage) using an `enum` and branching logic.
- **Data Access Object (DAO) Pattern:** `FileHandler` encapsulates all logic required to persist and retrieve data entities from the filesystem, hiding CSV parsing details from the business logic.

5. Data Flow (Checkout Process)

1. User requests checkout via CLI.
2. `Main` calls `Cart.checkout()` .
3. `Cart` verifies stock for all items one final time.
4. `Cart` deducts stock from `Product` objects.
5. `Cart` builds receipt string and clears internal items list.

6. `Main` prints receipt and calls `FileHandler.saveProducts()` to write modified `Product` data to `products.csv`.